

完整软件过程定义文档

课程：软件过程与管理
学院：四川大学软件学院
学期：2026 年春
作者：请填写姓名 / 学号 / 小组成员
提交格式：Markdown
提交渠道：GitHub

四、改进后的完整软件过程定义文档

4.1 过程概述

改进后的软件过程是一套面向智能化开发趋势的人机协同软件过程。该过程以传统 SDLC 为基础，覆盖需求分析、系统设计、编码实现、测试验证、部署发布、运维监控和持续改进七个阶段。与原有过程相比，新过程在关键节点引入 AI 辅助活动，并增加 AI 输出审核、AI 风险控制、AI 贡献度度量和 AI 使用记录。

该过程遵循以下原则。

- 人工负责原则：所有正式交付物必须由明确责任人确认，AI 不承担最终责任。
- 可验证原则：AI 输出必须通过评审、测试、静态检查或数据核对验证。
- 最小必要原则：只在收益明确、风险可控的活动中引入 AI。
- 数据安全原则：输入 AI 工具前必须进行敏感信息识别和脱敏。
- 可追溯原则：记录 AI 工具、提示词、输出摘要、修改记录和审核人。
- 持续改进原则：通过度量数据定期评估 AI 对效率和质量的真实影响。

4.2 过程总流程

```
flowchart TD
    A[项目启动] --> B[需求分析]
    B --> B1[AI 辅助需求整理]
    B1 --> C[需求评审]
    C --> D[系统设计]
    D --> D1[AI 辅助架构与接口建议]
    D1 --> E[设计评审]
    E --> F[编码实现]
    F --> F1[AI 辅助编码与重构]
    F1 --> F2[AI 代码预审]
    F2 --> G[人工代码审查]
    G --> H[测试验证]
    H --> H1[AI 辅助测试用例生成]
    H1 --> H2[测试执行与缺陷管理]
    H2 --> I[部署发布]
    I --> J[运维监控]
    J --> J1[AI 辅助异常分析]
    J1 --> K[复盘与持续改进]
    K --> B
```

4.3 阶段一：项目启动

4.3.1 阶段目标 项目启动阶段的目标是明确项目背景、范围、目标、干系人、约束条件、初步风险和资源计划。对于引入 AI 的项目，还需要明确 AI 工具使用边界、数据安全要求和过程度量方式。

4.3.2 主要活动

活动	输入	输出	责任角色
明确项目目标	项目任务书、课程要求	项目章程	项目经理

活动	输入	输出	责任角色
确定过程裁剪方案	标准软件过程、项目规模	过程裁剪说明	项目经理、质量负责人
制定 AI 使用规范	工具清单、数据安全要求	AI 使用规范	AI 过程负责人
制定进度计划	阶段目标、人员分工	项目计划	项目经理
建立风险台账	项目约束、技术不确定性	初始风险台账	项目经理

4.3.3 质量标准 项目章程必须说明项目目标、范围、关键交付物和验收标准。AI 使用规范必须说明允许使用的工具、禁止输入的信息、输出审核方式和责任边界。项目计划必须至少覆盖四周节点：调研分析、方案设计、原型验证、文档整合。

4.4 阶段二：需求分析

4.4.1 阶段目标 需求分析阶段的目标是形成清晰、完整、可验证的需求规格说明书，并建立需求追踪矩阵。AI 在该阶段的作用是辅助整理需求材料、生成用户故事、补全验收标准和识别潜在冲突。

4.4.2 主要活动

活动	AI 参与方式	输出
收集原始需求	不直接参与，避免污染访谈事实	访谈记录、业务资料
整理需求材料	AI 根据访谈记录生成摘要和问题清单	需求摘要、待确认问题
生成用户故事	AI 将业务目标转换为用户故事初稿	用户故事列表
补全验收标准	AI 为每条用户故事生成可测试验收标准	验收标准清单
检查一致性	AI 辅助识别术语不一致、流程冲突和缺失异常路径	需求冲突清单
需求评审	人工确认，不由 AI 代替	需求规格说明书

4.4.3 需求阶段质量门禁 需求文档进入设计阶段前，必须满足以下条件。

- 每条核心需求都有唯一编号。
- 每条功能需求都有验收标准。
- 关键业务流程包含正常路径和异常路径。
- 需求冲突和未决问题已有处理结论。
- AI 生成内容已经由需求负责人确认。
- 需求追踪矩阵已经建立。

4.5 阶段三：系统设计

4.5.1 阶段目标 系统设计阶段的目标是将需求转化为系统结构、模块划分、接口契约、数据模型和非功能设计。AI 在该阶段主要用于方案扩展、接口草拟和风险提示。

4.5.2 主要活动

活动	AI 参与方式	人工控制点
架构备选方案生成	AI 根据需求和约束生成候选架构	架构师判断是否符合项目约束
模块划分	AI 建议模块边界和职责	团队评审耦合度和可维护性
接口契约草拟	AI 生成接口字段、状态码和异常响应	开发负责人确认可实现性
数据模型检查	AI 提示字段遗漏和关系不清	数据负责人确认业务含义
风险分析	AI 生成安全、性能、扩展性风险清单	项目组评估概率和影响

4.5.3 设计评审清单

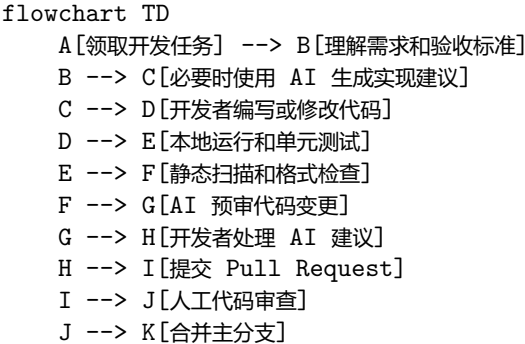
- 架构是否满足核心业务场景。
- 模块职责是否单一且边界清晰。

- 接口输入、输出、异常响应是否完整。
- 非功能需求是否有设计支撑。
- AI 建议是否被逐条采纳、拒绝或修改。
- 被拒绝的 AI 建议是否说明原因。

4.6 阶段四：编码实现

4.6.1 阶段目标 编码实现阶段的目标是按照设计文档和编码规范完成可运行、可测试、可维护的代码。AI 可以用于代码补全、样板代码生成、重构建议、注释生成和代码预审，但开发者必须对最终提交内容负责。

4.6.2 活动流程



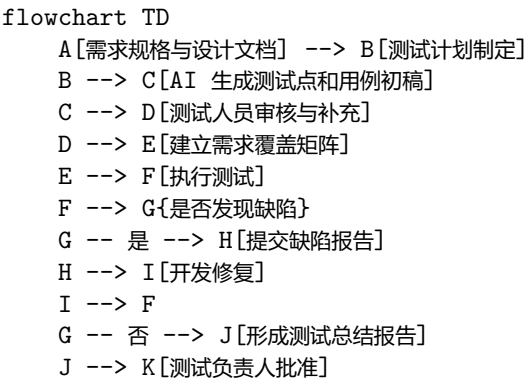
4.6.3 编码规范要求

- AI 生成代码必须符合项目语言、框架和目录结构。
- 复杂逻辑必须由开发者补充必要注释。
- 禁止提交未运行、未理解、不可解释的 AI 生成代码。
- 代码提交必须关联需求编号或缺陷编号。
- Pull Request 必须包含变更说明、测试说明和风险说明。

4.7 阶段五：测试验证

4.7.1 阶段目标 测试验证阶段的目标是验证软件是否满足需求规格和质量标准。AI 在该阶段用于测试点提取、测试用例生成、边界条件补充、自动化脚本草拟和测试报告摘要。

4.7.2 测试活动流程



4.7.3 测试交付物

交付物	内容	质量要求
测试计划	测试范围、策略、资源、进度、风险	覆盖核心功能和高风险模块

交付物	内容	质量要求
测试用例集	编号、测试点、步骤、预期结果、优先级	可执行、可追踪、无明显重复
需求覆盖矩阵	需求编号与测试用例映射	核心需求覆盖率达到 95% 以上
缺陷报告	缺陷描述、复现步骤、严重程度、截图或日志	可复现、定位信息完整
测试总结报告	执行结果、缺陷统计、遗留风险、结论	数据真实，结论明确

4.8 阶段六：部署发布

4.8.1 阶段目标 部署发布阶段的目标是将通过测试的软件版本安全、稳定地发布到目标环境。AI 可以辅助生成发布说明、检查部署清单和总结变更风险，但发布批准仍由项目负责人和运维负责人完成。

4.8.2 发布门禁

- 所有高严重级别缺陷已关闭或有明确延期批准。
- 自动化测试通过率满足发布标准。
- 发布包版本号、配置项和数据库脚本已确认。
- 回滚方案已经准备。
- 发布说明经过人工确认。
- AI 生成的变更摘要已与真实提交记录核对。

4.9 阶段七：运维监控

4.9.1 阶段目标 运维监控阶段的目标是保障系统上线后的可用性、性能和安全性。AI 在该阶段用于异常检测、日志摘要、告警聚合、根因分析建议和故障复盘初稿生成。

4.9.2 运维流程

flowchart LR

```

A[系统日志/指标/链路追踪] --> B[监控平台采集]
B --> C[AI 异常聚类与摘要]
C --> D[运维人员确认]
D --> E{是否故障}
E -- 否 --> F[关闭或调整告警规则]
E -- 是 --> G[定位根因并修复]
G --> H[发布补丁或配置变更]
H --> I[故障复盘]
I --> J[知识库更新]

```

4.9.3 运维质量指标

指标	含义	目标
MTTA	平均告警响应时间	持续下降
MTTR	平均故障恢复时间	持续下降
告警误报率	无需处理告警占比	控制在可接受范围
根因定位时间	从告警到确认根因的耗时	相比原过程减少 30%
知识库复用率	已有故障经验被复用的比例	持续提升

4.10 交付物清单

阶段	交付物	是否包含 AI 内容	审核人
项目启动	项目章程、过程裁剪说明、AI 使用规范	是	项目经理
需求分析	需求规格说明书、用户故事、需求追踪矩阵	是	需求负责人
系统设计	概要设计、接口契约、架构风险清单	是	架构师

阶段	交付物	是否包含 AI 内容	审核人
编码实现	源代码、单元测试、PR 记录、代码审查记录	是	开发负责人
测试验证	测试计划、测试用例、覆盖矩阵、测试报告	是	测试负责人
部署发布	发布说明、部署清单、回滚方案	可选	运维负责人
运维监控	监控报告、故障复盘、知识库条目	是	运维负责人

4.11 质量保证机制

改进后的过程采用多层质量保证机制。第一层是自动化检查，包括格式检查、单元测试、静态扫描、接口契约测试和 CI 构建。第二层是 AI 辅助检查，包括需求一致性提示、代码预审、测试用例补全和日志摘要。第三层是人工评审，包括需求评审、设计评审、代码审查、测试报告评审和发布评审。第四层是度量反馈，包括 AI 采纳率、修改率、缺陷前移率、测试覆盖率和发布后故障率。

这种机制的关键在于：AI 可以增加检查面，但不能取消人工责任；AI 可以提出建议，但不能绕过质量门禁；AI 可以加快文档和代码初稿生成，但不能替代验证和确认。

4.12 过程度量指标

维度	指标	数据来源	改进目标
效率	需求初稿耗时、测试用例设计耗时、代码审查准备耗时	工时记录、工具日志	减少重复性工作耗时
质量	缺陷密度、需求遗漏数、测试覆盖率、PR 返工次数	缺陷系统、测试报告、代码仓库	降低返工和遗漏
AI 有效性	AI 建议采纳率、AI 输出修改率、无效建议比例	AI 使用记录、审核记录	提高 AI 输出质量
安全	敏感信息输入次数、脱敏合规率、安全扫描问题数	安全检查记录	避免数据泄露
协作	评审周期、阻塞问题数、知识库复用次数	项目管理工具、复盘记录	提高团队协作效率

4.13 AI 使用记录模板

字段	说明
使用日期	记录 AI 工具使用时间
使用人员	记录责任人
工具名称	如 ChatGPT、GitHub Copilot、Kimi、通义千问等
使用场景	需求生成、测试用例生成、代码审查等
输入材料	说明输入内容是否脱敏
输出摘要	概括 AI 生成内容
采纳情况	全部采纳、部分采纳、未采纳
人工修改说明	记录主要修改点
审核人	负责确认输出的人

4.14 持续改进机制

每个迭代结束后，项目组应召开过程复盘会议，重点讨论 AI 节点的真实效果。复盘内容包括：哪些 AI 场景节省了时间，哪些输出质量不足，哪些提示词需要改进，是否出现数据安全问题，是否对交付质量产生负面影响。复盘结论应更新到提示词模板库、AI 使用规范和风险台账中。

持续改进遵循 PDCA 循环。

flowchart LR

```

    P[Plan 制定 AI 改进假设] --> D[Do 在迭代中试用]
    D --> C[Check 收集效率和质量数据]
    C --> A[Act 固化有效做法并修正问题]
    A --> P
  
```

4.15 总结

改进后的软件过程在传统 SDLC 的基础上引入 AI 辅助节点，能够提高需求整理、测试设计、代码审查和运维分析等环节的效率。但 AI 带来的价值只有在过程管理中才能稳定释放。本文设计的过程强调人工负责、质量门禁、数据安全和度量反馈，避免将 AI 简化为“自动生成答案”的工具。通过原型验证可以看出，AI 辅助测试用例生成能够显著降低测试设计总耗时，并提升需求覆盖率和异常路径覆盖度。因此，将 AI 纳入软件过程是可行的，但必须以可验证、可追溯、可控制为前提。

参考文献与资料

1. DORA, *Accelerate State of DevOps Report 2024*, Google Cloud, 2024. <https://dora.dev/research/2024/dora-report/>
2. Stack Overflow, *2024 Developer Survey: AI*, 2024. <https://survey.stackoverflow.co/2024/ai/>
3. GitHub Docs, *Using GitHub Copilot code review*, 2026 年访问. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
4. Haowei Cheng et al., *Generative AI for Requirements Engineering: A Systematic Literature Review*, arXiv:2409.06741, 2024. <https://arxiv.org/abs/2409.06741>
5. Junwei Liu et al., *Large Language Model-Based Agents for Software Engineering: A Survey*, arXiv:2409.02977, 2024. <https://arxiv.org/abs/2409.02977>
6. 中国信息通信研究院, 《智能化软件工程技术和应用要求 第 3 部分: 智能测试能力》, 2024。
7. GitHub Blog, *Survey reveals AI's impact on the developer experience*, 2024. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>